

MetricsRepository — Complete Reader Guide

1. Purpose

MetricsRepository is the PostgreSQL data-access layer for API monitoring metrics.

It is responsible for:

- Writing/accumulating endpoint metrics.
- Reading time-series metrics.
- Finding top endpoints.
- Calculating overall monitoring statistics.
- Building parameterized SQL queries safely.
- Limiting query size and database execution time.

High-level flow:

```
API Traffic
  |
  v
Processor / Aggregator
  |
  v
MetricsRepository
  |
  +--> upsertEndpointMetrics()
  |
  v
PostgreSQL: endpoint_metrics
  |
  +--> getMetrics()
  +--> getTopEndpoints()
  +--> getOverallStats()
```

2. Constants

```
const MAX_LIMIT = 1000;
const QUERY_TIMEOUT_MS = 30000;
```

MAX_LIMIT

Maximum number of rows returned by paginated queries.

```
Requested limit = 5000
MAX_LIMIT      = 1000
```

```
Actual limit = 1000
```

This protects PostgreSQL from very large result requests.

QUERY_TIMEOUT_MS

30000 means a database query is allowed to run for approximately 30 seconds before PostgreSQL cancels it.

3. Class and Constructor

```
export class MetricsRepository extends BaseRepository {
  constructor({ logger: l, postgres: pg } = {}) {
    super({ logger: l })
    this.postgres = pg;
  }
}
```

The repository receives dependencies: logger and postgres. The logger is passed to BaseRepository, and the PostgreSQL client is stored as this.postgres = pg;. Dependency injection makes the repository easier to test because a test can provide a mock PostgreSQL client.

4. Database Table Concept

The repository expects a table similar to:

endpoint_metrics

client_id
service_name
endpoint
method
total_hits
error_hits
avg_latency
min_latency
max_latency
time_bucket
updated_at

A useful interpretation is:

One row = One client + One service + One endpoint + One HTTP method +
One time bucket

The unique key for the upsert is: (client_id, service_name, endpoint, method, time_bucket).

5. upsertEndpointMetrics()

Purpose: This method inserts metrics into PostgreSQL. If the same client/service/endpoint/method/time bucket already exists, it updates the existing row instead of creating another row. This is called an **UPSERT**:

```

INSERT
|
|  +-- no conflict --> create row
|
|  +-- conflict ----> update existing row

```

Method input:

```
async upsertEndpointMetrics(metricsData)
```

Example:

```

{
  clientId: 100,
  serviceName: "payment",
  endpoint: "/payment",
  method: "POST",
  totalHits: 10,
  errorHits: 2,
  avgLatency: 150,
  minLatency: 80,
  maxLatency: 300,
  timeBucket: "2026-08-30 10:00:00"
}

```

6. Destructuring metricsData

```

const {
  clientId,
  serviceName,
  endpoint,
  method,
  totalHits,
  errorHits,
  avgLatency,
  minLatency,
  maxLatency,
  timeBucket,
} = metricsData;

```

This converts one object into individual variables (e.g., metricsData.clientId -> clientId).

7. INSERT Query

The query starts with:

```

INSERT INTO endpoint_metrics (
  client_id,
  service_name,

```

```

        endpoint,
        method,
        total_hits,
        error_hits,
        avg_latency,
        min_latency,
        max_latency,
        time_bucket
    )
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)

```

The placeholders correspond to: \$1 -> clientId, \$2 -> serviceName, \$3 -> endpoint, \$4 -> method, \$5 -> totalHits, \$6 -> errorHits, \$7 -> avgLatency, \$8 -> minLatency, \$9 -> maxLatency, \$10 -> timeBucket.

8. ON CONFLICT

```

ON CONFLICT (
    client_id,
    service_name,
    endpoint,
    method,
    time_bucket
)

```

This says: *If another row already has the same combination of these five values, do not create a duplicate row.* Instead of failing, PostgreSQL executes DO UPDATE SET.

9. EXCLUDED

Inside PostgreSQL's ON CONFLICT DO UPDATE, EXCLUDED represents the row that we attempted to insert. For example, EXCLUDED.total_hits means the total_hits value from the new incoming record.

10. Accumulating Metrics (total_hits & error_hits)

```

total_hits = endpoint_metrics.total_hits + EXCLUDED.total_hits
error_hits = endpoint_metrics.error_hits + EXCLUDED.error_hits

```

This allows the processor to continuously accumulate traffic and error counters without losing previous counts.

11. Average Latency Formula & Important Bug

The intended formula for weighted average latency is:

```
(
```

```

        (endpoint_metrics.avg_latency * endpoint_metrics.total_hits)
    +
    (EXCLUDED.avg_latency * EXCLUDED.total_hits)
)
/
(
    endpoint_metrics.total_hits
    +
    EXCLUDED.total_hits
)

```

Bug in original implementation:

The original code missing proper parentheses allowed operator precedence to perform division before addition:

```

avg_latency = (
    (endpoint_metrics.avg_latency * endpoint_metrics.total_hits)
    + (EXCLUDED.avg_latency * EXCLUDED.total_hits)
    / (endpoint_metrics.total_hits + EXCLUDED.total_hits)
)

```

The complete numerator must be wrapped in parentheses before division.

12. MIN / MAX Latency & updated_at

- **MIN Latency:** Uses LEAST(endpoint_metrics.min_latency, EXCLUDED.min_latency) to keep the fastest request time.
- **MAX Latency:** Uses GREATEST(endpoint_metrics.max_latency, EXCLUDED.max_latency) to keep the slowest request time.
- **updated_at:** Uses CURRENT_TIMESTAMP to log when an update occurs for auditing and operational monitoring.

13. Query & Filtering Methods Summary

Method	Purpose	Grouping Strategy
upsertEndpointMetrics()	Insert/update metric bucket	Unique key (client + service + endpoint + method + time_bucket)
getMetrics()	Time-series metrics retrieval	GROUP BY service_name, endpoint, method, time_bucket
getTopEndpoints()	Highest traffic endpoints	GROUP BY service_name, endpoint, method (No time_bucket)
getOverallStats()	Dashboard overall summary	No GROUP BY (Aggregates full filtered set)
_query()	Internal PostgreSQL query handler	N/A (Applies timeouts and parameter values)

14. Internal Query Execution (`_query`)

`_query()` standardizes PostgreSQL executions across the repository by handling target client resolution (allowing transaction client passing), validating client existence, and enforcing execution bounds:

```
return target.query({
  text: sql,
  values: params,
  statement_timeout: QUERY_TIMEOUT_MS
});
```

15. Final Summary

MetricsRepository serves as a safe, optimized, and dynamic data access layer supporting write-heavy upsert patterns and distinct aggregate read patterns for operational monitoring dashboards.